

The Interference Budget: what a fixed-size memory actually trades away

DataForge 2026 · Pathway Track · one-page concept summary. Team Code Rebels: Mohammed Amaan Khan (Leader), Arslan Alam. Concept: synaptic plasticity as short-term memory (additive fast-weight / linear-attention state). Artifact: an interactive explainer with a live toy substrate. Audience: a data scientist who knows softmax attention.

THE DESIGN PRESSURE

Softmax attention stores the past losslessly: a KV cache of N tokens, scanned per query. Cost grows with the sequence, which is the constraint driving the entire post-Transformer programme. Linear attention removes the softmax so the sum over past tokens can be reassociated into a single **state matrix** updated once per token (Katharopoulos et al., arXiv:2006.16236), equivalently, a **fast-weight** memory written by outer products (Schlag et al., arXiv:2102.11174). Read cost becomes constant. The usual framing stops there, and it is where most explanations go wrong.

Central claim. The saving is not free and is not primarily about compute. An additive state stores every association in *one shared matrix*, so a query returns the value it asked for *plus* a crosstalk term $\sum_{j \neq i} v_j (k_j \cdot k_i)$. The binding constraint is therefore *key overlap*, not storage. Nothing ever runs out of space; what degrades is separability.

WHAT OUR ARTIFACT DEMONSTRATES, LIVE

The explainer runs a deliberately small associative memory (8 symbols × d neurons) in the browser and lets the learner falsify two consequences of that claim. **First:** accuracy is a function of the ratio N/d, not of N. Over seeds 1-8 we measure 100%, 98%, 89%, 70% at N/d = 0.25, 0.5, 1.0, 2.0, and the *same* four numbers at d = 64, 128 and 256. **Second, and less obvious:** sparsifying the keys helps only if the code is also non-negative. Sparsifying a *signed* code from 100% to 3% active moves accuracy 94.5% → 94.4% (nothing: the shared coordinates cancel in sign, so overlap stays pinned near 1/√d). Sparsifying a *non-negative* code over the same range moves it 20.3% → 90.8%, with mean overlap falling 0.93 → 0.03. Sparsity and non-negativity are not two independent optimisations; each is what makes the other viable.

WHERE BDH SITS

Dragon Hatchling (BDH; Kosowski, Uznański, Chorowski, Stamirowska, Bartoszkiewicz, arXiv:2509.26507) reformulates attention as synaptic plasticity: a state σ the authors interpret directly as synapse weights between neuron pairs, written Hebbian-style as the model reads. Its normalisation-free update is $\sigma_t := (\sigma_{t-1} + y_{t-1} x_t^T) U$ (Eq. 7), with closed form $\sigma_{t-1} = \sum_{\tau < t} y_\tau x_\tau^T U^{t-\tau}$ (Eq. 16), **a purely additive, damped sum of outer products with no error-correcting term**. BDH therefore inherits exactly the crosstalk problem above and must control it structurally. It does: BDH-GPU is described as "a specific kind of ReLU-lowrank feed-forward network, and a linear attention mechanism, which both operate in the same neuron dimension n, using positive activation vectors" (Claim 2), with reported sparsity around 5%. That is the pairing our artifact isolates, and it explains a second BDH result for free, since a synapse driven by few neurons can only encode few things, which is the structural precondition for the monosemantic synapses the paper reports.

THE LANDSCAPE

SYSTEM	STATE UPDATE	HOW IT CONTROLS INTERFERENCE	COST OF THAT CHOICE
Softmax + KV	append	none needed, keys stay separate	state and read grow with N
Linear attn.	additive	nothing structural	degrades with key overlap
GLA / Mamba	gated / selective	data-dependent decay: forget selectively	old facts fade whether or not stale
DeltaNet	delta rule	subtract the stale value when writing	sequential dependency in the update
BDH	additive + damping	sparse non-negative addressing	needs very wide n; damping still hand-set

Read the last two rows against each other: DeltaNet fixes the *write*, BDH fixes the *code*. Our artifact lets a learner run both repairs on the same matrix and see that neither one enlarges it.

EVIDENCE, CLASSIFIED HONESTLY

Reproduced by us, live: only the general mechanism, ratio-invariance, the sparsity × non-negativity interaction, and the delta rule's advantage on repeated keys. Deterministic, seeded, and re-runnable with node `verify.mjs`. **Reported by BDH's authors, not reproduced:** the ~5% sparsity figure (measured 4.0-7.5% on a *synthetic letter task*, not natural language); monosemanticity (*two hand-picked synapses* on a Europarl model, Mann-Whitney U); heavy-tailed connectivity (one head of one layer, **no power-law exponent is fitted anywhere in the paper**, so "scale-free with exponent α" is an overstatement we avoid). Language results are character-level Europarl loss against a TransformerXL-style baseline, and the parity claim belongs to the *gated* BDH-GPU' variant, not a benchmark sweep against GPT-2. **BDH-CQ** (arXiv:2608.09888) is the later sibling doing recurrent latent reasoning with no parameter updates at inference; its 24.25% pass@1 / 29.5% pass@2 on ARC-AGI-1 public eval at ~\$0.0007/task from a 150M model are *developer-reported*, not leaderboard-verified (an AWS post quotes 29.2% for the same result). Its role in *this* concept is peripheral and we do not inflate it.

THE MOST IMPORTANT LIMITATION

Ours: keys here are random, not learned, so our curves are a pessimistic lower bound, a trained model can actively decorrelate its keys, and how much that buys is not something this artifact measures. BDH's, in its own words: an additive state needs damping "to avoid overwhelming the model with noise from stale context", and the paper supplies RoPE/ALiBi-style damping rather than a principled criterion. **That is the open question this concept leaves standing:** gating (GLA, Mamba), error-correction (DeltaNet) and sparse positive coding (BDH) are three uncoordinated answers to one problem, and no published evaluation isolates them against each other on equal footing. The comparison would be worth running.

Continue from here: BDH arXiv:2509.26507 §5.2 and Eq. 7/16 · Schlag et al. arXiv:2102.11174 §3 (fast weights and capacity) · Yang et al. arXiv:2406.06484 §2 (delta rule, key collisions) · Yang et al. arXiv:2312.06635 (gating) · Behrouz et al. arXiv:2501.00663 (test-time memory) · Katharopoulos et al. arXiv:2006.16236. · **Artifact + source:** github.com/amaancoderx/DataForge-2026-Code-Rebels- Prose CC BY 4.0, code MIT. AI assistance disclosed in README.